Relatório de Auditoria de Segurança — SolarSync

ConectSol · 24/09/2026 · cinco categorias: isolamento, permissão no navegador, IDOR, chaves expostas e XSS

Escopo auditado

Repositório	Conteúdo
solarsync (branch Plan, c4f8a66)	API Spring Boot: 10 controllers / 70 handlers, services, 18 migrations, application*.properties, compose.yaml, Dockerfile, deploy/ (compose, Caddyfile, scripts), scripts/, docs/, testes e histórico git (24 commits)
solarsync-front (branch solarsync-homologacao-projetos, 2da1971)	Interface React: src/ (54 arquivos), index.html, Dockerfile, dist/ compilado e histórico git (13 commits)

Nota metodológica

Categoria	Como foi mapeada para a stack
Stack detectada	Backend Java 21 + Spring Boot 4.1 (WebMVC, Security, Data JPA/Hibernate, Flyway) sobre PostgreSQL 16; autenticação JWT HS256 própria (resource server do Spring) e RBAC via @PreAuthorize em meta-anotações. Frontend React 18 + TypeScript + Vite + Tailwind (repositório solarsync-front). Deploy: Dockerfile multi-stage, Docker Compose e Caddy (TLS + proxy reverso). Não há CI, Helm nem Terraform no repositório.
1. Banco sem tranca	Sistema de uma única empresa (single-tenant), sem Supabase/RLS. O mecanismo de isolamento é apenas o papel do usuário: por decisão documentada (CLAUDE.md §4 e §8, itens 4 e 5), ANALISTA vê todos os clientes. Verificou-se que não há coluna de dono que deveria filtrar consultas e que listagens, buscas e agregações do dashboard só são alcançáveis autenticado.
2. Permissão no navegador	Cruzados os gates do frontend (Sidebar, UsuariosModule, temPapel) com os 70 handlers REST e com as guardas de hierarquia do UsuarioService.
3. IDOR	Percorridos todos os 70 handlers dos 10 controllers. Como não há posse por usuário, acesso por id sem checagem de dono é o comportamento especificado; a análise concentrou-se onde há hierarquia de objeto (usuários e conta técnica) e na origem do autor da auditoria.
4. Chaves expostas	Busca por padrões de segredo e strings de alta entropia na árvore e em todo o histórico git dos dois repositórios (24 + 13 commits), em configs, compose, Dockerfile, scripts, documentação, testes e no bundle dist/ do frontend; revisão de defaults ${VAR:-...} e da validação de startup.
5. XSS	Busca por dangerouslySetInnerHTML, innerHTML, insertAdjacentHTML, document.write, eval, new Function, href/src dinâmicos, window.open e bibliotecas de markdown no frontend; no backend, geração de HTML (e-mails, templates) e cabeçalhos de segurança do proxy.

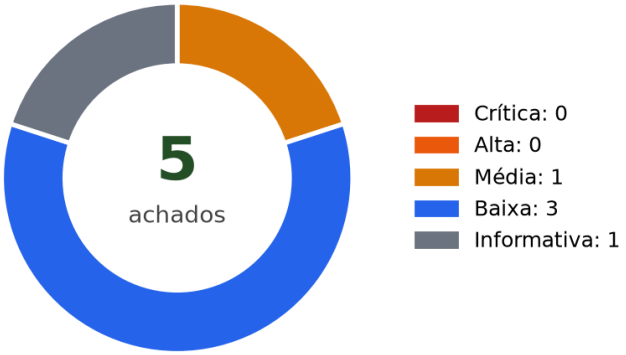
Só entram achados verificados no código. Categorias que não se aplicam à stack são declaradas como tal, com a evidência de por quê.

Resumo executivo

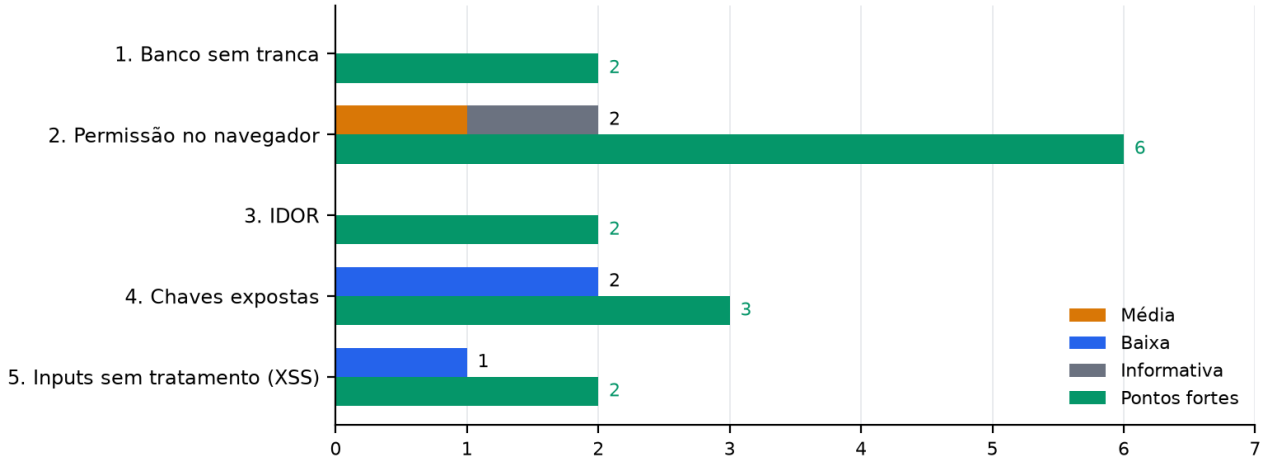
A auditoria encontrou **5 achados**: nenhum crítico ou alto, 1 de severidade média, 3 baixos e 1 informativo. O controle de acesso é feito no servidor em todos os 70 handlers, não há segredo real nem no código nem no histórico git, e o frontend não tem nenhum sink de XSS. As categorias 1 (isolamento por dono) e 3 (IDOR) não se aplicam no sentido clássico: o sistema é single-tenant e todo papel lê tudo por decisão de negócio documentada. O único achado com impacto real é a conta técnica de integração, que um GESTOR consegue reativar e usar para gravar ações em nome da automação.

0	0	1	3	1	15
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes

Achados por severidade



Achados e pontos fortes por categoria



Pontos fortes e pontos fracos

Pontos fortes (verificados)

Cat.	O que está protegido	Evidência
2	● **Todo handler sensível tem guarda no servidor**	Dos 70 handlers, 67 têm @PodeLer/@PodeEscrever/@SomenteAdministrador/@GerenciaUsuarios; os 3 restantes são /api/auth/login e /refresh (públicos por natureza) e /api/auth/eu (autenticado pelo anyRequest().authenticated() — SecurityConfig.java:65).
2	● **Negação por padrão e conversor de papéis correto**	SecurityConfig.java:52-65 (só login, refresh, swagger e health são permitAll) e SecurityConfig.java:81-90 (claim papeis com prefixo ROLE_, coberto por PendenciaControllerRbacTest).
2	● **Hierarquia de papéis replicada no servidor**	UsuarioService.java:114-127: GESTOR não concede ADMINISTRADOR nem altera um administrador; UsuarioService.java:93-95: ninguém desativa a própria conta. É o espelho de UsuariosModule.tsx:140-144, não só UI.
2	● **Exclusão só por ADMINISTRADOR, e a tela nem oferece**	Os 7 DELETE usam @SomenteAdministrador; o frontend não tem nenhum botão de excluir; corrigir-status (ProjetoController.java:204-205) também é @SomenteAdministrador.
3	● **Autor da auditoria sempre do token, nunca do corpo**	UsuarioAutenticado.java:20-27 lê o id do subject do JWT; os controllers passam usuario.id() aos services. Nenhum request DTO carrega o autor da mudança.
3	● **Trilhas sem endpoint de escrita**	historico_status e email_coelba só são escritos por listener/job; não há POST/PUT para elas.
1	● **Consultas parametrizadas, inclusive o SQL nativo**	DashboardRepository.java:31-53 interpola só nomes de coluna constantes; valores entram por setParameter (linhas 233-235 e 340-343). O restante usa JPA Specifications.
2	● **Tokens bem separados e renovação relendo o banco**	TipoTokenValidator impede refresh token como bearer; emissor validado; segredo HS256 exige 32+ bytes (JwtConfig.java:54-59); RenovacaoTokenService.java:34-39 recusa inativo.
2	● **Login resistente a enumeração e a força bruta**	LoginSenhaService: mesma exceção para todos os motivos e hash falso para igualar o tempo; ControleDeTentativasDeLogin recusa antes do BCrypt (10/e-mail, 60/origem em 15 min).
4	● **Nenhum segredo real na árvore nem no histórico**	24 commits do backend e 13 do frontend varridos: só placeholders (token-de-teste, GOCSPX-...). .gitignore cobre .env, .env.*, config/ e *.pem; config/application.properties local nunca foi commitado.
4	● **Deploy sem defaults de segredo**	deploy/compose.yaml:30 usa ${POSTGRES_PASSWORD:?...}; JWT, senha inicial, Nectar e Gmail vêm vazios em .env.example e deploy/.env.example.
4	● **Bundle do frontend limpo**	dist/assets/*.js não contém chave; a única variável VITE_ é VITE_API_URL (endereço, não segredo).
5	● **Nenhum sink de XSS no frontend**	Zero ocorrências de dangerouslySetInnerHTML, innerHTML, insertAdjacentHTML, eval, new Function, href/src dinâmico ou lib de markdown em src/; tudo passa pelo escape do React. Dados vindos de e-mail e do CRM são exibidos como texto.
5	● **Backend não gera HTML**	Nenhum e-mail é enviado (Gmail é gmail.readonly) e as respostas são JSON/ProblemDetail; server.error.include-* = never em application-prod.properties.

Cat.	O que está protegido	Evidência
1	● **Banco e API fora da internet**	deploy/compose.yaml: só o Caddy publica porta; Postgres com scram-sha-256; container não roda como root (Dockerfile); /actuator responde 404 de fora (Caddyfile:34-36); Swagger desligado em prod.

Categorias sem achado

1. Banco sem tranca — não se aplica. Não há multi-tenancy nem posse de registro por usuário: é uma empresa só, e ANALISTA vê todos os clientes por decisão documentada (CLAUDE.md §4, §8 itens 4 e 5). Não existe filtro por dono a faltar. O que protege os dados é a autenticação obrigatória e o banco fora da internet.

3. IDOR — não se aplica. Os 70 handlers buscam por id sem checar dono, e isso é o comportamento especificado (qualquer papel lê e edita qualquer etapa; só ADMINISTRADOR apaga). Onde há hierarquia de objeto — usuários — ela é verificada no servidor, com a exceção reportada em A-01. O autor gravado na auditoria nunca vem do corpo da requisição.

Pontos fracos (riscos centrais)

MÉDIA — **A-01 · GESTOR reativa e assume a conta técnica de integração** — Exige uma conta GESTOR (papel confiável). Não há escalada para ADMINISTRADOR. O impacto é de integridade e repúdio: aprovações, reprovações e transições feitas pela conta ficam no historico_status como "Integração automática", indistinguíveis da automação do Gmail/Nectar — exatamente a trilha usada para responder "por que este projeto foi reprovado".

BAIXA — **A-02 · Sem validação de startup para segredos em produção** — Depende de configuração errada no servidor. A senha fraca do administrador fica exposta ao login público, mitigado pelo limite de 10 falhas/15 min por e-mail. O segredo ausente não abre acesso, mas desloga todos a cada restart sem nada acusar.

BAIXA — **A-03 · Senha fixa do Postgres de dev com porta publicada em todas as interfaces** — Só em máquina de desenvolvimento com o container de pé e rede compartilhada (Wi-Fi do escritório, por exemplo) sem firewall bloqueando. Afeta o banco de dev, que pode receber a importação da planilha real (docs/importacao).

BAIXA — **A-04 · Sem Content-Security-Policy e tokens em localStorage** — Não explorável hoje: exige primeiro uma falha de XSS, que não existe no código atual. O refresh token roubado vale 8 h ou até o usuário ser desativado.

INFORMATIVA — **A-05 · Desativação e rebaixamento de papel valem só após até 15 min** — Janela de até 15 min para um usuário recém-desligado continuar agindo com o papel antigo.

Observações fora das cinco categorias

`ControleDeTentativasDeLogin.java:100-104` — Usa o primeiro endereço de X-Forwarded-For. Seguro hoje porque o Caddy substitui o cabeçalho vindo de cliente não confiável. Ao ligar o Cloudflare (item 24), configurar trusted_proxies do Cloudflare no Caddy — senão o limite por origem passa a contar o IP do nó do Cloudflare.

`scripts/obter-refresh-token-gmail.ps1:48-60` — Fluxo OAuth de loopback sem parâmetro state/PKCE. Uso único e local; risco baixo.

`deploy/backup.sh:31-36` — O dump completo do banco é gravado com o umask padrão. Considerar umask 077 no script.

Achados detalhados por categoria

Nos caminhos abaixo, …/ abrevia `src/main/java/com/conectsol/solarsync/`. Os caminhos `solarsync-front/` são do repositório do frontend.

1. Banco sem tranca

Nenhum achado — ver "Categorias sem achado" e os pontos fortes.

2. Permissão no navegador

Severidade	Arquivo:linha	Descrição
MÉDIA	…/auth/UsuarioService.java:120-127 …/auth/UsuarioService.java:81-98 src/main/resources/db/migration/V13__integracoes_nectar_e_gmail.sql:68-69 solarsync-front/src/components/modules/UsuariosModule.tsx:140, 230, 396	**A-01 · GESTOR reativa e assume a conta técnica de integração.** A única fronteira da gestão de usuários no servidor é "o alvo é ADMINISTRADOR?". A conta técnica integracao@conectsol.com não tem papel nenhum, então passa pela guarda: um GESTOR pode chamar POST /api/usuarios/{id}/ativar, POST /{id}/senha e PUT /{id} com papeis=[GESTOR], e depois entrar por ela. A tela oferece o botão "Reativar" para essa conta (podeMexerEm só exclui administradores). A mesma rota permite ao GESTOR redefinir a senha de outro GESTOR e agir em nome dele, e nenhuma operação de gestão de usuários gera trilha de auditoria (o UsuarioService não publica evento).
INFORMATIVA	…/auth/jwt/TokenService.java:62 …/auth/jwt/JwtConfig.java:75	**A-05 · Desativação e rebaixamento de papel valem só após até 15 min.** O servidor confia nos papéis e na existência do usuário que estão no JWT até ele expirar. Desativar alguém ou tirar o papel GESTOR corta a renovação na hora, mas o access token já emitido continua valendo até 15 minutos. Decisão documentada e aceita (CLAUDE.md §10).

A-01 — trecho, explorabilidade e correção

```
// UsuarioService.java:121-126
private static void exigirPoderSobre(Usuario alvo, UsuarioAutenticado autor) {
    boolean alvoEhAdmin = alvo.getPapeis().stream()
            .anyMatch(papel -> papel.getNome() == NomePapel.ADMINISTRADOR);
    if (alvoEhAdmin && !autor.tem(NomePapel.ADMINISTRADOR)) { throw ... }
}
-- V13:68-69
INSERT INTO usuario (nome, email, ativo)
VALUES ('Integração automática', 'integracao@conectsol.com', FALSE);
```

Explorabilidade: Exige uma conta GESTOR (papel confiável). Não há escalada para ADMINISTRADOR. O impacto é de integridade e repúdio: aprovações, reprovações e transições feitas pela conta ficam no historico_status como "Integração automática", indistinguíveis da automação do Gmail/Nectar — exatamente a trilha usada para responder "por que este projeto foi reprovado".

Correção: Marcar a conta técnica como conta de sistema (coluna ou comparação com UsuarioIntegracao.EMAIL) e recusar, no UsuarioService, ativar, definir senha e editar papéis dela para qualquer papel; recusar também no LoginSenhaService. Esconder a conta da tela de Usuários. Registrar em auditoria quem redefiniu a senha, ativou/desativou ou mudou papéis de quem.

A-05 — trecho, explorabilidade e correção

```
// TokenService.java:62 — papéis gravados no access token
.claim(CLAIM_PAPEIS, papeis)
// JwtConfig.java:75 — validação só criptográfica, sem reler o usuário
return decoderDoTipo(chaveJwt, propriedades, TokenService.TIPO_ACCESS);
```

Explorabilidade: Janela de até 15 min para um usuário recém-desligado continuar agindo com o papel antigo.

Correção: Aceitável como está. Se a janela incomodar: versão de token por usuário (token_version no JWT, conferida contra cache) ou encurtar o access token.

3. IDOR

Nenhum achado — ver "Categorias sem achado" e os pontos fortes.

4. Chaves expostas

Severidade	Arquivo:linha	Descrição

BAIXA	`src/main/resources/application.properties:16`
	`…/auth/jwt/JwtConfig.java:44-52`
	`…/auth/AdminBootstrap.java:38, 64`
		A-02 · Sem validação de startup para segredos em produção. Não há default público de segredo — o default é vazio e o fallback é aleatório, o que não é explorável. O problema é a ausência de fail-fast: com SPRING_PROFILES_ACTIVE=prod a aplicação sobe sem SOLARSYNC_JWT_SEGREDO apenas com um WARN. E a senha inicial do administrador é aplicada sem a política de 8+ caracteres que o DefinirSenhaRequest exige no resto do sistema: SOLARSYNC_ADMIN_SENHA_INICIAL=123 é aceita.

BAIXA	`compose.yaml:7-9`	**A-03 · Senha fixa do Postgres de dev com porta publicada em todas as interfaces.** O compose de desenvolvimento (usado pelo suporte a Docker Compose do Spring Boot) tem senha fixa commitada e publica a 5432 numa porta efêmera do host em 0.0.0.0. O primeiro commit (66a4492) trazia POSTGRES_PASSWORD=secret.

A-02 — trecho, explorabilidade e correção

```
# application.properties:16
solarsync.jwt.segredo=${SOLARSYNC_JWT_SEGREDO:}
// JwtConfig.java:45-51
if (segredo == null || segredo.isBlank()) {
    log.warn("solarsync.jwt.segredo ausente: gerando segredo aleatório...");
    ... new SecureRandom().nextBytes(aleatorio);
// AdminBootstrap.java:64 (sem nenhuma regra de tamanho)
admin.setSenhaHash(passwordEncoder.encode(senhaInicial));
```

Explorabilidade: Depende de configuração errada no servidor. A senha fraca do administrador fica exposta ao login público, mitigado pelo limite de 10 falhas/15 min por e-mail. O segredo ausente não abre acesso, mas desloga todos a cada restart sem nada acusar.

Correção: No perfil prod, falhar o boot se solarsync.jwt.segredo estiver vazio. Exigir no AdminBootstrap o mesmo mínimo do DefinirSenhaRequest (idealmente 12+) e recusar o boot com senha fraca.

A-03 — trecho, explorabilidade e correção

```
# compose.yaml:4-9
environment:
  - 'POSTGRES_PASSWORD=solarsync_dev'
ports:
  - '5432'
```

Explorabilidade: Só em máquina de desenvolvimento com o container de pé e rede compartilhada (Wi-Fi do escritório, por exemplo) sem firewall bloqueando. Afeta o banco de dev, que pode receber a importação da planilha real (docs/importacao).

Correção: Publicar só no loopback ('127.0.0.1::5432') e ler a senha de variável com default de dev. Não reaproveitar solarsync_dev em nenhum outro ambiente.

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
BAIXA	`deploy/Caddyfile:53-61` `solarsync-front/src/api/client.ts:28-38`	**A-04 · Sem Content-Security-Policy e tokens em localStorage.** Não foi encontrado nenhum sink de XSS no frontend (ver pontos fortes). O achado é de defesa em profundidade: o access token e o refresh token de 8 h ficam em localStorage, legíveis por qualquer script da origem, e o proxy não envia CSP. Uma única injeção futura, ou uma dependência comprometida, entregaria as duas credenciais sem nada que a contivesse.

A-04 — trecho, explorabilidade e correção

```
# Caddyfile:53-61 — não há Content-Security-Policy
header {
  Strict-Transport-Security "max-age=31536000"
  X-Content-Type-Options "nosniff"
  X-Frame-Options "DENY"
  Referrer-Policy "same-origin"
// client.ts:32-33
localStorage.setItem(CHAVE_ACCESS, accessToken);
localStorage.setItem(CHAVE_REFRESH, refreshToken);
```

Explorabilidade: Não explorável hoje: exige primeiro uma falha de XSS, que não existe no código atual. O refresh token roubado vale 8 h ou até o usuário ser desativado.

Correção: Enviar CSP pelo Caddy: default-src 'self'; script-src 'self'; style-src 'self' https://fonts.googleapis.com; font-src https://fonts.gstatic.com; connect-src 'self'; img-src 'self' data:; object-src 'none'; base-uri 'self'; frame-ancestors 'none'. Validar na interface (começar com Content-Security-Policy-Report-Only). A médio prazo, levar o refresh token para cookie HttpOnly/SameSite=Strict no mesmo domínio.

Recomendações priorizadas

Prioridade	Achado	Ação	Quando / por quê
P1	**A-01**	Blindar a conta técnica de integração e auditar a gestão de usuários	Antes da homologação com a equipe: é o único achado que corrompe a trilha de auditoria.
P2	**A-02**	Fail-fast de segredos no perfil prod e política na senha inicial do admin	Antes do primeiro deploy (item 23 do roadmap).
P2	**A-04**	Content-Security-Policy no Caddy (começar em Report-Only)	Barato, e contém o dano do único cenário que exporia os tokens.
P3	**A-03**	Postgres de dev só no loopback e sem senha fixa commitada	Higiene.
P3	**A-05**	Reavaliar a janela de 15 min de revogação	Só se o desligamento imediato virar requisito.
P3	**—**	trusted_proxies do Cloudflare no Caddy ao ligar o WAF	Junto com o item 24 do roadmap.

ISSUES PARA O GITHUB

Texto completo de cada issue em Markdown, pronto para copiar e colar. Os achados A-02 e A-03 foram agrupados numa issue só (mesmo tema: segredos e defaults). O mesmo conteúdo está em docs/security-audit/issues-github.md.

Issue 1 — [Segurança] GESTOR consegue reativar e assumir a conta técnica de integração

```
--- ISSUE 1 ---
Título: [Segurança] GESTOR consegue reativar e assumir a conta técnica de integração
Labels: security, severity:medium
Achados: A-01

## Descrição
A única fronteira da gestão de usuários no servidor é "o alvo é ADMINISTRADOR?". A conta técnica
`integracao@conectsol.com` (semeada pela V13 com `ativo = false`, sem senha e sem papel) passa por essa
guarda. Um GESTOR pode reativá-la, definir uma senha, dar-lhe um papel e entrar por ela. A tela de
Usuários mostra o botão "Reativar" para essa conta.

A mesma rota permite ao GESTOR redefinir a senha de outro GESTOR e agir em nome dele, e nenhuma operação
de gestão de usuários gera trilha de auditoria.

## Por que é explorável
1. `POST /api/usuarios/{idIntegracao}/ativar`
2. `POST /api/usuarios/{idIntegracao}/senha` com `{"senha": "..."}`
3. `PUT /api/usuarios/{idIntegracao}` com `papeis: ["GESTOR"]`
4. `POST /api/auth/login` como `integracao@conectsol.com`

Toda transição feita depois disso fica no `historico_status` como "Integração automática", indistinguível
da automação do Gmail/Nectar.

## Evidência
- `src/main/java/com/conectsol/solarsync/auth/UsuarioService.java:120-127`
```java
private static void exigirPoderSobre(Usuario alvo, UsuarioAutenticado autor) {
    boolean alvoEhAdmin = alvo.getPapeis().stream()
            .anyMatch(papel -> papel.getNome() == NomePapel.ADMINISTRADOR);
    if (alvoEhAdmin && !autor.tem(NomePapel.ADMINISTRADOR)) {
        throw new AccessDeniedException("Só um ADMINISTRADOR altera outro ADMINISTRADOR");
    }
}
```
- `src/main/java/com/conectsol/solarsync/auth/UsuarioService.java:81-98` (`definirSenha`,
`alterarAtivacao` só chamam a guarda acima)
- `src/main/resources/db/migration/V13__integracoes_nectar_e_gmail.sql:68-69`
- `solarsync-front/src/components/modules/UsuariosModule.tsx:140` (`podeMexerEm` só exclui
administradores), `:230`, `:396` (botão Reativar)

Impacto
Integridade e repúdio: aprovações, reprovações e mudanças de status atribuídas à automação, e a mesma
coisa em nome de outro gestor. Sem escalada para ADMINISTRADOR.

Sugestão de correção
- Tratar a conta técnica como conta de sistema (coluna `conta_sistema` ou comparação com
`UsuarioIntegracao.EMAIL`) e recusar com 403, no `UsuarioService`, `ativar`, `definirSenha` e `atualizar`
para ela, qualquer que seja o papel do autor.
- Recusar o login dessa conta no `LoginSenhaService`, mesmo com senha definida.
- Esconder a conta da tela de Usuários.
- Registrar em auditoria quem redefiniu a senha, ativou/desativou ou mudou os papéis de quem.

Critérios de aceite
- [ ] GESTOR e ADMINISTRADOR recebem 403 ao ativar, definir senha ou editar papéis de
`integracao@conectsol.com`
- [ ] Login com `integracao@conectsol.com` falha mesmo se `senha_hash` estiver preenchido
- [ ] A conta não aparece na listagem da tela de Usuários
- [ ] Redefinir senha, ativar/desativar e mudar papéis gravam autor, alvo e instante numa trilha
consultável
- [ ] Teste de integração cobrindo os quatro passos da exploração acima
```

```
--- FIM ISSUE 1 ---
```

Issue 2 — [Segurança] Validar segredos no boot e remover credenciais fixas de dev

```
--- ISSUE 2 ---
Título: [Segurança] Validar segredos no boot e remover credenciais fixas de dev
Labels: security, severity:low
Achados: A-02, A-03

## Descrição
Nenhum segredo real está no repositório, mas três pontos deixam a segurança depender de alguém lembrar de
configurar certo:

1. Com `SPRING_PROFILES_ACTIVE=prod`, a aplicação sobe sem `SOLARSYNC_JWT_SEGREDO`, só com um WARN,
usando um segredo aleatório.
2. `SOLARSYNC_ADMIN_SENHA_INICIAL` é aplicada ao administrador sem a política de tamanho mínimo que o
`DefinirSenhaRequest` exige no resto do sistema (`123` é aceita).
3. O `compose.yaml` de desenvolvimento tem a senha `solarsync_dev` commitada e publica a 5432 em todas as
interfaces do host.

## Por que é explorável
- (2) Uma senha fraca de administrador fica exposta ao login público, mitigado só pelo limite de 10
falhas/15 min por e-mail.
- (3) Em rede compartilhada, qualquer máquina alcança o Postgres de dev com a senha pública.
- (1) Não abre acesso, mas desloga todos a cada restart sem nada acusar.

## Evidência
- `src/main/resources/application.properties:16` — `solarsync.jwt.segredo=${SOLARSYNC_JWT_SEGREDO:}`
- `src/main/java/com/conectsol/solarsync/auth/jwt/JwtConfig.java:44-52` — fallback para segredo aleatório
- `src/main/java/com/conectsol/solarsync/auth/AdminBootstrap.java:64` —
`admin.setSenhaHash(passwordEncoder.encode(senhaInicial));`
- `compose.yaml:7-9`
```yaml
- 'POSTGRES_PASSWORD=solarsync_dev'
ports:
  - '5432'
```

Impacto
Baixo: depende de configuração errada no servidor ou de rede de dev exposta.

Sugestão de correção
- No perfil `prod`, falhar o boot se `solarsync.jwt.segredo` estiver vazio.
- No `AdminBootstrap`, exigir o mesmo mínimo do `DefinirSenhaRequest` (idealmente 12+) e recusar o boot
com senha fraca.
- No `compose.yaml`, publicar como `'127.0.0.1::5432'` e ler a senha de
`${POSTGRES_PASSWORD_DEV:-solarsync_dev}`.

Critérios de aceite
- [ ] `SPRING_PROFILES_ACTIVE=prod` sem `SOLARSYNC_JWT_SEGREDO` falha no boot com mensagem clara
- [ ] `SOLARSYNC_ADMIN_SENHA_INICIAL` com menos que o mínimo falha no boot
- [ ] `docker compose up` de dev não escuta a 5432 fora do loopback (`docker port` mostra 127.0.0.1)
- [ ] Testes cobrindo os dois fail-fast
--- FIM ISSUE 2 ---
```

Issue 3 — [Segurança] Adicionar Content-Security-Policy e reduzir a exposição dos tokens

```
--- ISSUE 3 ---
Título: [Segurança] Adicionar Content-Security-Policy e reduzir a exposição dos tokens
Labels: security, severity:low
Achados: A-04

## Descrição
Não há nenhum sink de XSS no frontend hoje. Mas o access token e o refresh token (8 h) ficam em
`localStorage`, legíveis por qualquer script da origem, e o Caddy não envia `Content-Security-Policy`.
Uma injeção futura, ou uma dependência comprometida, entregaria as duas credenciais sem nada que a
contivesse.
```

```
## Por que é explorável
Não é explorável sozinho: exige antes uma falha de XSS. É defesa em profundidade para limitar o estrago
desse cenário.

## Evidência
- `deploy/Caddyfile:53-61` — bloco `header` sem CSP
- `solarsync-front/src/api/client.ts:28-38`
```ts
localStorage.setItem(CHAVE_ACCESS, accessToken);
localStorage.setItem(CHAVE_REFRESH, refreshToken);
```

Impacto
Roubo de sessão por até 8 h (ou até o usuário ser desativado) caso surja um XSS.

Sugestão de correção
Adicionar ao bloco `header` do Caddyfile, primeiro como `Content-Security-Policy-Report-Only`:
```
default-src 'self'; script-src 'self'; style-src 'self' https://fonts.googleapis.com;
font-src https://fonts.gstatic.com; img-src 'self' data:; connect-src 'self';
object-src 'none'; base-uri 'self'; frame-ancestors 'none'
```
Depois de validar a interface, trocar para `Content-Security-Policy`. A médio prazo, avaliar levar o
refresh token para cookie `HttpOnly; Secure; SameSite=Strict` (mesmo domínio já elimina CORS).

Critérios de aceite
- [ ] Resposta de `/` e de `/api/*` traz `Content-Security-Policy` com `script-src 'self'` e
`frame-ancestors 'none'`
- [ ] Todas as telas funcionam sem violação no console do navegador
- [ ] Um `<script>` inline injetado em teste manual é bloqueado pelo navegador
--- FIM ISSUE 3 ---
```

Issue 4 — [Segurança] Reavaliar a janela de 15 min para revogar acesso e papel

```
--- ISSUE 4 ---
Título: [Segurança] Reavaliar a janela de 15 min para revogar acesso e papel
Labels: security, severity:informational
Achados: A-05

## Descrição
Os papéis ficam gravados no access token e o servidor não relê o usuário a cada requisição. Desativar
alguém ou tirar o papel GESTOR corta a renovação na hora, mas o access token já emitido vale até 15
minutos. Decisão documentada e aceita (CLAUDE.md §10); esta issue existe para ser reavaliada, não
necessariamente corrigida.

## Evidência
- `src/main/java/com/conectsol/solarsync/auth/jwt/TokenService.java:62` — `.claim(CLAIM_PAPEIS, papeis)`
- `src/main/java/com/conectsol/solarsync/auth/jwt/JwtConfig.java:75` — validação só criptográfica

## Impacto
Janela de até 15 min para um usuário recém-desligado agir com o papel antigo.

## Sugestão de correção
Se virar requisito: um `token_version` por usuário no JWT, conferido contra um cache de curta duração,
incrementado ao desativar ou ao mudar papéis.

## Critérios de aceite
- [ ] Decisão registrada (manter ou implementar)
- [ ] Se implementar: requisição com token de usuário recém-desativado responde 401 imediatamente
--- FIM ISSUE 4 ---
```